



PERTEMUAN V

FORECASTING

5.1 TUJUAN PEMBELAJARAN

- A. Mahasiswa dapat menerapkan metode forecasting berbasis *machine learning*, meliputi Neural Network, K-Nearest Neighbors (KNN), Decision Tree, Support Vector Regression (SVR), dan Random Forest, untuk memprediksi nilai periode berikutnya berdasarkan data historis
- B. Mahasiswa dapat melakukan analisis dan perbandingan performa model menggunakan metrik evaluasi *Root Mean Squared Error* (RMSE) dan *Pearson Correlation Coefficient*
- C. Mahasiswa dapat menentukan model forecasting yang paling akurat berdasarkan hasil evaluasi menggunakan RMSE dan *Pearson Correlation Coefficient*

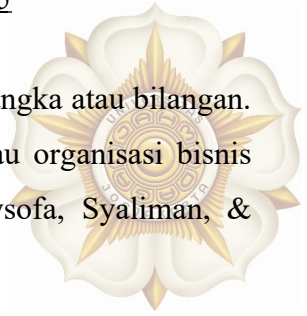
5.2 DASAR TEORI

- **Supervised Learning**

Supervised learning merupakan salah satu teknik *machine learning* yang menggunakan dataset (*data training*) berlabel (*labeled data*) untuk melatih mesin, sehingga mampu mengidentifikasi label input berdasarkan fitur yang dimiliki dan melakukan prediksi atau klasifikasi. Algoritma yang termasuk dalam teknik *supervised learning* antara lain Decision Tree, K-Nearest Neighbor (KNN), Naïve Bayes, Regresi, dan Support Vector Machine (SVM) (Retnoningsih & Pramudita, 2020). Pada algoritma *Supervised Learning*, sistem diberikan training data set berupa informasi masukan dan keluaran yang diinginkan, sehingga sistem akan mempelajari berdasarkan data yang telah ada. Sistem akan mencari pola dari data set, kemudian pola itu akan dijadikan sebagai acuan untuk kumpulan data berikutnya (Santoso, Abijono, & Anggreini, 2021).

- **Forecasting**

Forecasting adalah proses memprediksi suatu keadaan di masa kini dan masa depan dengan menguji keadaan di masa lalu. Metode ini banyak digunakan untuk melakukan prediksi dengan berbagai algoritma (Vimala & Nugroho, 2022). Fungsi ramalan adalah membantu pengambilan keputusan berdasarkan pertimbangan terhadap apa yang akan terjadi saat keputusan tersebut dilaksanakan. Ramalan dapat bersifat kualitatif, yaitu tidak berbentuk angka, seperti perkiraan bahwa cuaca besok akan cerah. Selain itu, ramalan



juga bisa bersifat kuantitatif, yang berarti dinyatakan dalam bentuk angka atau bilangan. Peramalan merupakan bagian penting dalam setiap perusahaan atau organisasi bisnis karena berperan dalam pengambilan keputusan manajemen (Maysofa, Syaliman, & Sapriadi, 2023).

- **RMSE**

Root Mean Squared Error (RMSE) merupakan salah satu metrik evaluasi yang digunakan untuk mengukur tingkat kesalahan antara nilai aktual dan nilai hasil prediksi. RMSE dihitung dengan cara mengambil akar kuadrat dari rata-rata selisih kuadrat antara nilai aktual dan nilai prediksi. Semakin kecil nilai RMSE, semakin baik performa model karena menunjukkan bahwa model memiliki error yang rendah. RMSE banyak digunakan dalam peramalan (forecasting) karena sensitif terhadap error yang besar, sehingga mampu memberikan gambaran akurasi model secara lebih representatif (Chai & Draxler, 2014).

- **Pearson Correlation Coefficient**

Pearson Correlation Coefficient adalah ukuran statistik yang digunakan untuk menentukan tingkat hubungan linear antara dua variabel, yaitu nilai aktual dan nilai prediksi. Nilai koefisien berkisar antara -1 hingga 1 , di mana nilai mendekati 1 menunjukkan korelasi positif yang sangat kuat, nilai mendekati -1 menunjukkan korelasi negatif yang kuat, dan nilai mendekati 0 menunjukkan tidak adanya hubungan linear. Dalam konteks forecasting, Pearson Correlation digunakan untuk menilai seberapa baik pola prediksi mengikuti pola aktual. Semakin tinggi nilai korelasi, semakin baik model dalam menangkap pola data (Benesty et al., 2009).

5.3 ALAT DAN BAHAN

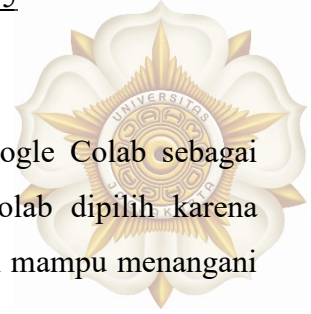
A. Perangkat Keras

1. Laptop dengan minimal RAM 4 GB
2. Koneksi internet stabil

B. Perangkat Lunak

1. Browser (Chrome, Edge, Firefox, dsb)
2. Google Colab / Jupyter Notebook
3. Library ML (pandas, numpy, scikit-learn, matplotlib, dsb)

5.4 LANGKAH PERCOBAAN



1. Persiapan IDE

Mahasiswa menyiapkan lingkungan kerja dengan membuka Google Colab sebagai platform utama untuk menjalankan eksperimen *forecasting*. Colab dipilih karena mendukung seluruh library machine learning yang diperlukan dan mampu menangani komputasi tanpa membebani perangkat mahasiswa.

2. Import Library

Setelah lingkungan siap, mahasiswa mengimpor seluruh *library* yang diperlukan untuk melakukan preprocessing, visualisasi, dan proses *forecasting*. *Library* utama yang digunakan meliputi:

1. pandas dan numpy untuk membaca, memanipulasi, dan mengelola data.
2. matplotlib dan seaborn untuk membuat visualisasi data, seperti tren deret waktu dan grafik hasil prediksi.
3. MeanSquaredError / RMSE dan *Pearson correlation* untuk evaluasi performa model.
4. MLPRegressor, KNeighborsRegressor, DecisionTreeRegressor, dan RandomForestRegressor untuk melakukan pemodelan forecasting dengan berbagai algoritma *supervised learning*.
5. StandardScaler atau *library* pendukung lain apabila dibutuhkan untuk normalisasi atau transformasi data.

```
[ ] # Import Library

import numpy as np
import pandas as pd
from math import sqrt
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
import math
from sklearn.metrics import mean_squared_error
from numpy import array
from sklearn.neural_network import MLPRegressor
from sklearn.neighbors import KNeighborsRegressor
from scipy.stats import pearsonr
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVC
from sklearn.svm import SVR
from xgboost import XGBClassifier

from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.metrics import mean_squared_log_error
from numpy import array
from scipy.stats import kurtosis, skew
```

Gambar 5.4.1 Impor Pustaka



3. Import Dataset

Mahasiswa membuat dataset <https://www.kaggle.com/datasets/lakshmi25npathi/bike-sharing-dataset> yang dapat diakses melalui Bike Sharing Dataset. Dataset ini berisi data peminjaman sepeda per hari, termasuk tanggal, kondisi cuaca, dan total jumlah peminjaman. Pada tahap ini mahasiswa:

1. Mengimpor dataset dan menampilkan 5 baris pertama untuk memastikan dataset terbaca dengan benar.

```
# Read Dataset
import pandas as pd
df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/bikesharing_day.csv')
df.head()
```

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
0	1	1/1/2011	1	0	1	0	6	0	2	0.344167	0.363625	0.805833	0.160446	331	654	985
1	2	1/2/2011	1	0	1	0	0	0	2	0.363478	0.353739	0.696087	0.248539	131	670	801
2	3	1/3/2011	1	0	1	0	1	1	1	0.196364	0.189405	0.437273	0.248309	120	1229	1349
3	4	1/4/2011	1	0	1	0	2	1	1	0.200000	0.212122	0.590435	0.160296	108	1454	1562
4	5	1/5/2011	1	0	1	0	3	1	1	0.226957	0.229270	0.436957	0.186900	82	1518	1600

Gambar 5.4.2 Pembacaan Dataset

2. Melakukan konversi kolom tanggal ke format datetime

```
# Change Data Format
df_ori = df
df_ori['date'] = pd.to_datetime(df_ori['dteday'])
df_ori['cnt'].iloc[:10]
```

	cnt
0	985
1	801
2	1349
3	1562
4	1600

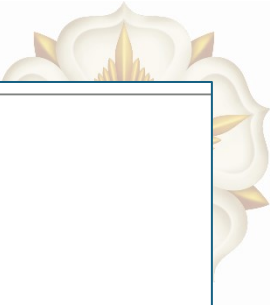
Gambar 5.4.3 Konversi tanggal ke format datetime

4. Membentuk Date Time Series

Data time series membutuhkan pembentukan input dan output secara berurutan. Oleh karena itu, mahasiswa membuat:

1. Input (X): nilai cnt selama 7 hari sebelumnya,
2. Output (y): jumlah peminjaman sepeda pada hari ke-8.

Pembentukan ini dilakukan dengan teknik *sliding window*, yaitu menggeser jendela data dari hari ke hari untuk menghasilkan pasangan input–output.



```
[ ] # Function Definition Sliding Window

def split_sequences(sequences, n_steps_in, n_steps_out):
    X, y = list(), list()
    for i in range(len(sequences)):
        # find the end of this pattern
        end_ix = i + n_steps_in
        out_end_ix = end_ix + n_steps_out

        # check if we are beyond the dataset
        if out_end_ix > len(sequences):
            break

        # gather input and output parts of the pattern
        seq_x, seq_y = sequences[i:end_ix, :-1], sequences[out_end_ix - 1, -1]
        X.append(seq_x)
        y.append(seq_y)
    return array(X), array(y)
```

Gambar 5.4.4 Fungsi sliding window

5. Menambahkan Fitur Statistik

Agar model memiliki informasi tambahan, mahasiswa melakukan feature engineering dengan menambahkan fitur-fitur statistik pada setiap jendela data, seperti:

1. Nilai minimum
2. Nilai maksimum
3. Selisih antara max-min
4. Rata-rata
5. Standar deviasi
6. Median
7. Kurtosis
8. Skewness



```
[ ] # Function Definition Statistic Feature

def stats_features(input_data):
    inp = list()
    for i in range(len(input_data)):
        inp2=list()
        inp2=input_data[i]
        min=float(np.min(inp2))
        max=float(np.max(inp2))
        diff=(max-min)
        std=float(np.std(inp2))
        mean=float(np.mean(inp2))
        median=float(np.median(inp2))
        kurt=float(kurtosis(inp2))
        sk=float(skew(inp2))
        inp2=np.append(inp2,min)
        inp2=np.append(inp2,max)
        inp2=np.append(inp2,diff)
        inp2=np.append(inp2,std)
        inp2=np.append(inp2,mean)
        inp2=np.append(inp2,median)
        inp2=np.append(inp2,kurt)
        inp2=np.append(inp2,sk)

        #print(list(inp2))
        inp=np.append(inp,inp2)

    inp=inp.reshape(len(input_data),-1)

    #print(inp)
    return inp
```

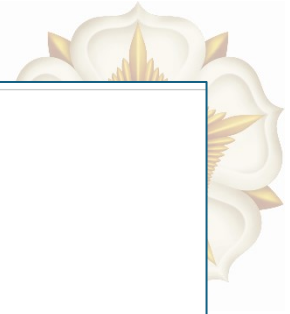
Gambar 5.4.5 Fungsi statistik

Fitur-fitur ini memberi gambaran kondisi distribusi data dalam jendela waktu tersebut, sehingga diharapkan dapat meningkatkan performa model.

6. Pemrosesan Data

Pada tahap ini dilakukan proses pengolahan data agar dapat digunakan sebagai input bagi model-model machine learning. Seluruh proses preprocessing dilaksanakan dalam satu rangkaian, meliputi:

1. Pembentukan Data Time Series Menggunakan *Sliding Window*
2. Pemisahan Data Training dan Testing



```
# Preprocessing

df_X = df_ori[['cnt', 'cnt']]
in_seq = df_X.astype(float).values
#out_seq = df_y.astype(float).values

#in_seq1 = in_seq.reshape(in_seq.shape[0], in_seq.shape[1])
#out_seq = out_seq.reshape((len(out_seq), 1))

#from numpy import hstack
#dataset = hstack((in_seq1, out_seq))

n_steps_in, n_steps_out = 7, 1
X, y = split_sequences(in_seq, n_steps_in, n_steps_out)

n_input = X.shape[1] * X.shape[2]
X = X.reshape((X.shape[0], n_input))

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, shuffle = False)
X_train = stats_features(X_train)
X_test = stats_features(X_test)
```

Gambar 5.4.6 Preprocessing Dataset

```
# Output from Data Transformation with Sliding Window Technique

in_seq

array([[ 985.,  985.],
       [ 801.,  801.],
       [1349., 1349.],
       ...,
       [1341., 1341.],
       [1796., 1796.],
       [2729., 2729.]])

# Input Features

X

... array([[ 985.,  801., 1349., ..., 1600., 1606., 1510.],
          [ 801., 1349., 1562., ..., 1606., 1510.,  959.],
          [1349., 1562., 1600., ..., 1510.,  959.,  822.],
          ...,
          [1749., 1787.,  920., ...,  441., 2114., 3095.],
          [1787.,  920., 1013., ..., 2114., 3095., 1341.],
          [ 920., 1013.,  441., ..., 3095., 1341., 1796.]])

# Data Shape

X.shape

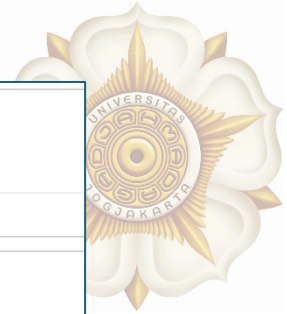
(724, 7)

# Array Input (Sample)

X [0]

array([ 985.,  801., 1349., 1562., 1600., 1606., 1510.]])
```

Gambar 5.4.7 Preprocessing Dataset



```
# Prediction Target

y[0]

np.float64(959.0)
```

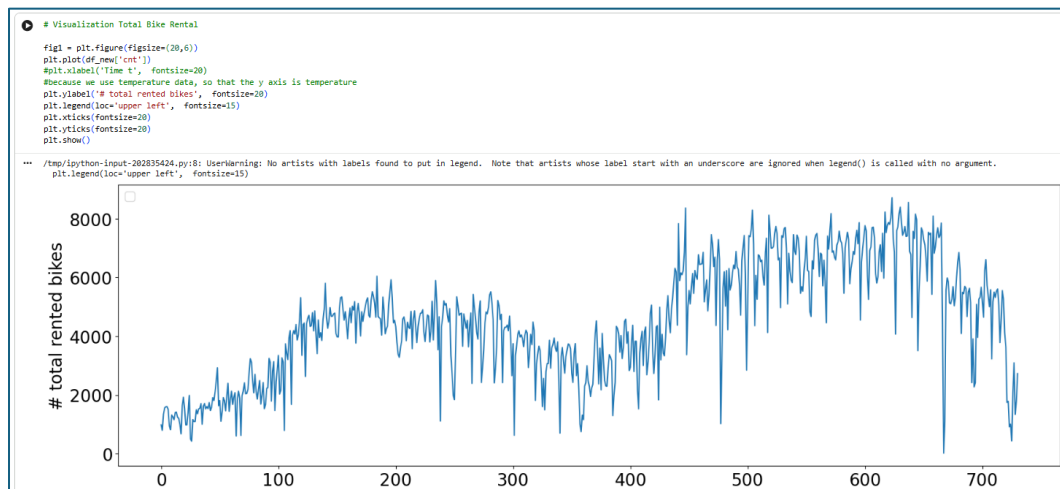
```
# Dataframe from Time Series Analysis

df_new=df_ori[['date','cnt']]
df_new.set_index('date')
```

date	cnt
2011-01-01	985
2011-01-02	801
2011-01-03	1349
2011-01-04	1562
2011-01-05	1600
...	...
2012-12-27	2114
2012-12-28	3095
2012-12-29	1341
2012-12-30	1796
2012-12-31	2729

731 rows x 1 columns

Gambar 5.4.8 Preprocessing Dataset

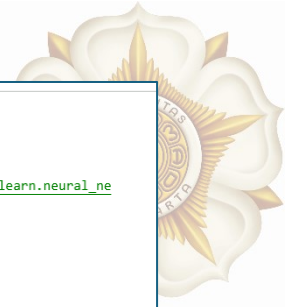


Gambar 5.4.9 Visualisasi Dataset

7. Pelatihan Model Machine Learning

Mahasiswa melatih empat model *supervised learning* untuk *forecasting*:

1. Multilayer Perceptron (MLP/Neural Network)



```
# Neural Network Model

def mlp (X_train, X_test, y_train, y_test):
    # mlp = multilayer perceptron / neural network for regression.
    # to setup parameter, please refer to = https://scikit-learn.org/stable/modules/generated/sklearn.neural\_ne
    mlp_model = MLPRegressor(random_state=42)
    mlp_model = MLPRegressor(hidden_layer_sizes=(100,100, ), max_iter=1000, random_state=42)
    # the model learning from training data
    mlp_model.fit(X_train, y_train)
    # get the prediction output
    y_pred = mlp_model.predict(X_test)
    y_pred = np.round(y_pred, 0)

    # get the root mean square error between prediction and real test data
    rmse = sqrt(mean_squared_error(y_test, y_pred))
    # get the pearson correlation
    corr, p_value = pearsonr(y_test, y_pred)
    # returning the output of RMSE, corr, and prediction result
    return rmse, corr, y_pred
```

Gambar 5.4.10 Fungsi Multilayer Perceptron

2. K-Nearest Neighbors (KNN)

```
# K-Nearest Neighbors Model

def knn (X_train, X_test, y_train, y_test):
    #k nearest neighbor for regression.
    #to setup the parameter please refer to : https://scikit-learn.org/stable/modules/generated/sklearn.neighb
    knn_model = KNeighborsRegressor()
    #the model is learning from training data
    knn_model.fit(X_train, y_train)
    #get the prediction output
    y_pred = knn_model.predict(X_test)
    y_pred = np.round(y_pred, 0)
    #get the root mean square error between prediction and real test data
    rmse = sqrt(mean_squared_error(y_test, y_pred))
    #get the pearson correlation
    corr, p_value = pearsonr(y_test, y_pred)
    #returning the output of RMSE, corr, and prediction result
    return rmse, corr, y_pred
```

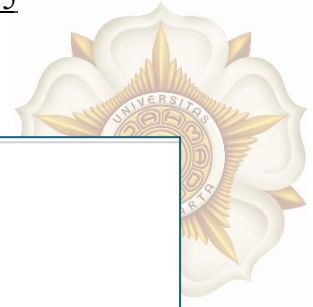
Gambar 5.4.11 Fungsi KNN

3. Decision Tree (DT)

```
# Decision Tree Model

def dt (X_train, X_test, y_train, y_test):
    model= DecisionTreeRegressor(random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_pred = np.round(y_pred, 0)
    rmse = sqrt(mean_squared_error(y_test, y_pred))
    #get the pearson correlation
    corr, p_value = pearsonr(y_test, y_pred)
    #returning the output of RMSE, corr, and prediction result
    return rmse, corr, y_pred
```

Gambar 5.4.12 Fungsi Decision Tree



4. Random Forest (RF)

```
# Random Forest Model

def rf (X_train, X_test, y_train, y_test):
    model=RandomForestRegressor(random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_pred = np.round(y_pred, 0)
    rmse = sqrt(mean_squared_error(y_test, y_pred))
    #get the pearson correlation
    corr, p_value = pearsonr(y_test, y_pred)
    #returning the output of RMSE, corr, and prediction result
    return rmse, corr, y_pred
```

Gambar 5.4.13 Fungsi Random Forest

Setiap model dilatih menggunakan data training, kemudian menghasilkan prediksi berdasarkan data testing. Tujuan langkah ini adalah membandingkan performa antar model.

8. Melakukan Prediksi

Setelah model dilatih, masing-masing model menghasilkan nilai prediksi jumlah peminjaman sepeda untuk periode berikutnya berdasarkan input data uji. Prediksi ini kemudian dibandingkan dengan data aktual.

```
# Call the Machine Learning Model Function

#calling the function mlp.
#returning rmse, pearson correlation, and prediction output
rmse_mlp, corr_mlp, y_pred_mlp = mlp(X_train, X_test, y_train, y_test)

#calling the function knn
#returning rmse, pearson correlation, and prediction output
rmse_knn, corr_knn, y_pred_knn = knn(X_train, X_test, y_train, y_test)
rmse_dt, corr_dt, y_pred_dt = dt(X_train, X_test, y_train, y_test)

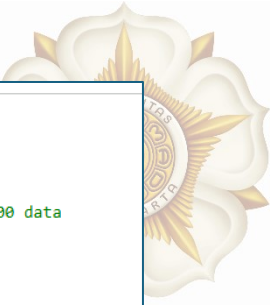
#rmse_svm, corr_svm, y_pred_svm = svm(X_train, X_test, y_train, y_test)
rmse_rf, corr_rf, y_pred_rf = rf(X_train, X_test, y_train, y_test)
```

Gambar 5.4.14 Lakukan prediksi

9. Visualisasi Hasil

Untuk melihat seberapa dekat pola prediksi dengan pola data asli mahasiswa dapat membuat visualisasi perbandingan antara:

1. Nilai aktual (y_{test}), dan
2. Prediksi model (terutama MLP sebagai model utama)



```
# Visualization

fig1 = plt.figure(figsize=(20,6))
# plotting the result
# because the total number of test is data is high, so we just print the first 100 data
# y_test[0:100] get the first 100 data from y_test or real test data.
# y_pred_mlp is the prediction result from MLP
# y_pred_knn is the prediction result from KNN

plt.plot(y_test, label='Real data')
# plt.plot(y_pred_knn, label='KNN')
# plt.plot(y_pred_dt, label='DT')
# plt.plot(y_pred_rf, label='RF')
plt.plot(y_pred_mlp, label='MLP')

#title
# pyplot.title('First 100 Test Data')
# the x axis is timestamp, with interval 1 day
plt.xlabel('Time t', fontsize=20)
# because we use total rented bikes, so that the y axis is rented bikes
plt.ylabel('rented bikes', fontsize=20)
plt.legend(loc='upper left', fontsize=15)
plt.xticks(fontsize=20)
plt.yticks(fontsize=20)
plt.show()
```

Gambar 5.4.15 Visualisasi hasil prediksi

10. Evaluasi Model

Evaluasi dilakukan menggunakan dua metrik utama:

1. *Root Mean Squared Error (RMSE)*
 - Mengukur seberapa besar kesalahan prediksi
 - Semakin kecil RMSE → model makin baik.
2. *Pearson Correlation Coefficient*
 - Mengukur hubungan linear antara nilai prediksi dan nilai aktual
 - Semakin mendekati 1 → prediksi semakin mengikuti pola aktual.

Mahasiswa diminta mencatat nilai RMSE dan Pearson dari seluruh model untuk dianalisis.



```
# Model Evaluation

# print out the RMSE , pearson correlation coefficient for MLP and KNN
# low RMSE is better
# high pearson correlation coefficient is better.
print('_____')
print('MLP')
print('RMSE : %.3f' %rmse_mlp)
print('Pearson correlation coefficient: %.3f' %corr_mlp)
print('_____')
print('KNN')
print('RMSE : %.3f' %rmse_knn)
print('Pearson correlation coefficient: %.3f' %corr_knn)
print('_____')
print('DT')
print('RMSE : %.3f' %rmse_dt)
print('Pearson correlation coefficient: %.3f' %corr_dt)
print('_____')
print('RF')
print('RMSE : %.3f' %rmse_rf)
print('Pearson correlation coefficient: %.3f' %corr_rf)
```

Gambar 5.4.16 Model Evaluation

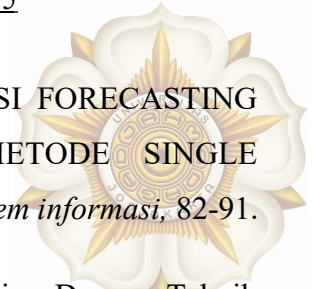
5.5 TUGAS DAN ANALISIS

1. Dengan menggunakan dataset ini <https://www.kaggle.com/datasets/timoboz/tesla-stock-data-from-2010-to-2020/data>
2. Buatlah model forecasting untuk memprediksi target (close) 2 hari kedepan, dengan menggunakan data close 7 hari sebelumnya!
3. Buatlah model forecasting untuk memprediksi target (close) 2 hari kedepan, dengan menggunakan data close dan open 7 hari sebelumnya!

5.6 REFERENSI

Benesty, J., Chen, J., Huang, Y., & Cohen, I. (2009). Pearson Correlation Coefficient. Dalam *Noise Reduction in Speech Processing* (pp. 1–4). Springer. https://doi.org/10.1007/978-3-642-00296-0_5

Chai, T., & Draxler, R. R. (2014). Root mean square error (RMSE) or mean absolute error (MAE)? Arguments against avoiding RMSE in the literature. *Geoscientific Model Development*, 7(3), 1247–1250. <https://doi.org/10.5194/gmd-7-1247-2014>



Maysofa, R., Syaliman, I., & Sapriadi, D. (2023). IMPLEMENTASI FORECASTING PADA PENJUALAN INAURA HAIR CARE DENGAN METODE SINGLE EXPONENTIAL SMOOTHING. *Jurnal Testing dan Implementasi Sistem informasi*, 82-91.

Retnoningsih, E., & Pramudita, A. (2020). Mengenal Machine Learning Dengan Teknik Supervised dan Unsupervised Learning Menggunakan Python. *BINA INSANI ICT JOURNAL*, 56-165.

Santoso, A., Abijono, H., & Anggreini, D. (2021). ALGORITMA SUPERVISED LEARNING DAN UNSUPERVISED LEARNING DALAM PENGOLAHAN DATA. *Jurnal Teknologi Terapan*, 315-318.

Vimala, T., & Nugroho, S. (2022). FORECASTING PENJUALAN OBAT MENGGUNAKAN METODE SINGLE, DOUBLE, DAN TRIPLE EXPONENTIAL SMOOTHING (STUDI KASUS : APOTEK MANDIRI MEDIKA). *Jurnal Penerapan Teknologi Informasi dan Komunikasi*, 90-99.